

1. Design Patterns in JavaScript

-  Teach:
 - Module Pattern (IIFE)
 - Revealing Module Pattern
 - Factory Function Pattern
 - Observer Pattern (basic pub-sub)
-  Confusion:
 - Module vs Factory
 - IIFE vs arrow function
-  Mindset:
 - Patterns are reusable ways to solve frequent design problems
-  Practice:
 - Build Notification Service using Observer pattern

2. Performance Optimization

-  Teach:
 - Debouncing and Throttling
 - Lazy Loading Images (with `IntersectionObserver`)
 - Code Splitting (intro level)
 - Avoiding unnecessary reflows and repaints
 - Memory leaks: timers, event listeners
-  Confusion:
 - Debounce vs Throttle timing behavior
-  Mindset:
 - UX is speed + responsiveness
-  Practice:
 - Live search with debounce
 - Scroll-based lazy loading list

3. Advanced Topics & Architecture Thinking

-  Teach:
 - Pure vs impure functions
 - Functional Programming basics (map/filter/reduce as pipeline)
 - Separation of concerns (DOM vs logic)
 - Custom utilities (e.g., own implementation of map, deep clone)
 - How JS works in browser (Event Loop, Web APIs, Call Stack)
-  Mindset:
 - Think in architecture, not features

1) Design Patterns in JavaScript — full explainer

What is a “design pattern” (mindset)

Patterns are **reusable solutions** to common problems — not concrete code you must copy, but proven approaches and trade-offs. They help you communicate ideas, structure code, and scale confidence when complexity grows.

Teach bullets (each with explanation + example)

Module Pattern (IIFE)

What: A pattern to create private state and expose an API. Classic JS solution before ES modules. Implemented using an Immediately Invoked Function Expression (IIFE) that returns an object with public methods.

Why: Encapsulates private variables (avoid global scope), exposes only chosen interface.

Example:

```
js

const Counter = (function () {
  let count = 0; // private
  function increment() { count++; }
  function get() { return count; }
  return { increment, get }; // public API
})();

Counter.increment();
console.log(Counter.get()); // 1
```

Confusion note: **IIFE** is about *how you create a closure/module*. An arrow function can be used inside an IIFE, but arrow functions behave differently for **this** and cannot be used as constructors. So “IIFE vs arrow function” is comparing *invocation style vs function type*.

Revealing Module Pattern

What: A variant of the module pattern that defines functions/variables privately and returns an object that maps public names to private functions (makes the public interface explicit).

Example:

js

```
const TodoModule = (function () {  
  let todos = [];  
  
  function add(task) { todos.push({ task, done: false }); }  
  function all() { return todos.slice(); }  
  
  // reveal smaller, explicit public API  
  return { add, all };  
})();
```

Factory Function Pattern

What: A function that creates and returns objects (without using `new` or classes). Useful for creating multiple similar objects with private state via closures.

Example:

js

```
function createPerson(name) {  
  let secret = 'shh';  
  return {  
    name,  
    greet() { console.log('Hi, I am', name); },  
    getSecret() { return secret; }  
  };  
}  
  
const p = createPerson('Dileep');  
p.greet(); // Hi, I am Dileep
```

Module vs Factory confusion:

- **Module (singleton-like):** IIFE returns one object representing a module (often single instance).
- **Factory:** returns a *new* object per call. Use factories when you need many instances with similar behavior.

Observer Pattern (basic pub-sub)

What: Decouples event producers and consumers. Subscribers register callbacks; publisher notifies them.

Use-case: Notification service, UI events, cross-component communication.

Basic implementation:

```
js

function createEventBus() {
  const subs = new Map();
  return {
    on(event, cb) {
      (subs.get(event) || subs.set(event, []).get(event)).push(cb);
    },
    off(event, cb) {
      const arr = subs.get(event) || [];
      subs.set(event, arr.filter(f => f !== cb));
    },
    emit(event, payload) {
      (subs.get(event) || []).slice().forEach(cb => cb(payload));
    }
  };
}

const bus = createEventBus();
bus.on('msg', data => console.log('got', data));
bus.emit('msg', { text: 'Hello' });
```

Practice: Build a *Notification Service* that other modules can subscribe to; include adding/removing listeners, and prioritizing messages.

Confusions to clear up (short)

- **Module vs Factory:** Module = single shared encapsulated object. Factory = function returning new objects.
- **IIFE vs arrow function:** IIFE describes immediate invocation pattern; arrow functions are a syntactic form and differ in `this`, `arguments`, and `new` behavior. You **can** use an arrow function inside an IIFE, but usually the IIFE is `((() => { ... } ))()` — arrow functions inside modules still follow arrow semantics.

Practice idea

- Implement a Notification Service using Observer pattern:
 - subscribe/unsubscribe
 - different channels (info, warn, error)
 - TTL for messages

- persistent store (optional)

2) Performance Optimization — complete guide

Mindset

UX = speed + responsiveness. Users care about perceived performance; making UI feel fast often matters more than micro-optimizations.

Teach bullets explained

Debouncing vs Throttling

Debounce: Delay a function until user *stops* triggering an event for N ms. Good for search input (wait until typing stops).

Throttle: Ensure a function runs at most once per interval. Good for scroll/resize handlers (limit frequency).

Examples:

```
js

// Debounce
function debounce(fn, wait) {
  let t;
  return function(...args) {
    clearTimeout(t);
    t = setTimeout(() => fn.apply(this, args), wait);
  };
}

// Throttle (timestamp version)
function throttle(fn, wait) {
  let last = 0;
  return function(...args) {
    const now = Date.now();
    if (now - last >= wait) {
      last = now;
      fn.apply(this, args);
    }
  };
}
```

Confusion: Debounce delays until input stops; throttle spaces out calls. They behave differently for the timing of the *first* and *last* calls — many implementations add options for leading/trailing behavior.

Lazy Loading Images (IntersectionObserver)

What: Load images only when they're near or in the viewport.

Why: Reduces initial load, saves bandwidth, improves time-to-interaction.

Example:

html

```

```

js

```
const io = new IntersectionObserver(entries => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      const img = entry.target;
      img.src = img.dataset.src;
      io.unobserve(img);
    }
  });
}, { rootMargin: '200px' });

document.querySelectorAll('img.lazy').forEach(img => io.observe(img));
```

Code splitting (intro)

What: Break a bundle into smaller chunks that load on demand (route-based or component-based). Makes initial bundle smaller and faster.

Conceptual notes:

- For frameworks: `import()` dynamic imports produce separate chunks.
- Useful for large libraries or rarely-used routes/features.

Minimal example (ES modules / bundlers):

js


```
button.addEventListener('click', async () => {  
  const { heavyFeature } = await import('./heavyFeature.js');  
  heavyFeature();  
});
```

Avoiding unnecessary reflows and repaints

Reflow (layout): When DOM geometry changes — expensive.

Repaint: When visual properties change but layout unaffected — cheaper than layout but still cost.

Tips:

- Batch DOM reads and writes separately (read → write pattern).
- Use `class` toggles instead of many inline style changes.
- Use `transform` and `opacity` for animations (GPU-friendly).
- Avoid layout thrashing (don't read layout values like `offsetHeight` repeatedly between writes).

Memory leaks: timers, event listeners

Common leak sources:

- DOM elements referenced from JS after removal.
- Timers (`setInterval`) never cleared.
- Event listeners added to elements or `window` and not removed.
- Closures capturing large objects.

Prevention:

- Remove listeners when components unmount.
- Use `WeakMap/WeakRef` for caches where appropriate.
- Clear timers on cleanup.

Practice tasks

- **Live search with debounce:** Implement an input that sends network requests only when user stops typing (debounce).
- **Scroll-based lazy loading list:** Use `IntersectionObserver` to load more items or images as user scrolls (infinite list).

3) Advanced Topics & Architecture Thinking

Mindset

Think in architecture, not features. Organize code for maintainability, testability, and change — not just immediate functionality.

Teach bullets explained

Pure vs Impure functions

- **Pure function:** Same inputs → same outputs, no side effects (e.g., `Math.max`-like). Easier to test and reason about.
- **Impure function:** Has side effects (mutates external state, does I/O) or depends on external state.

Example pure:

```
js  
  
function add(a,b){ return a+b; }
```

Impure example:

```
js  
  
let counter = 0;  
function inc() { counter++; } // impure: modifies external state
```

Why it matters: Pure functions enable safer refactorings, easier caching/memoization, and clearer reasoning.

Functional Programming basics (map/filter/reduce as pipeline)

Concepts:

- `map` transforms items.
- `filter` selects items.
- `reduce` aggregates.

Pipeline example:

```
js  
  
const numbers = [1,2,3,4,5];  
const result = numbers  
  .map(n => n*2)  
  .filter(n => n > 5)  
  .reduce((sum, n) => sum + n, 0);
```

Idea: Compose small pure functions for clarity and reuse.

Separation of concerns (DOM vs logic)

Principle: Keep UI rendering separate from business logic. Makes testing and maintenance easier.

Example: Have modules that compute data and return plain objects; separate view layer reads those objects and updates DOM.

Custom utilities (e.g., implement `map`, `deep clone`)

Implement your own `map` :

```
js

function myMap(arr, fn) {
  const out = [];
  for (let i = 0; i < arr.length; i++) out.push(fn(arr[i], i, arr));
  return out;
}
```

Deep clone (simple):

```
js

function deepClone(obj) {
  if (obj === null || typeof obj !== 'object') return obj;
  if (Array.isArray(obj)) return obj.map(deepClone);
  const out = {};
  for (const k in obj) {
    out[k] = deepClone(obj[k]);
  }
  return out;
}
```

Note: real-world deep clone needs to handle cycles, Dates, Regex, Map/Set — but the above is good for learning.

How JS works in browser (Event Loop, Web APIs, Call Stack)

Call stack: Where synchronous function calls are pushed and popped.

Web APIs: Browser-provided APIs (timers, DOM events, fetch) live outside JS engine. When you, e.g., `setTimeout`, the timer is handled by the Web API and its callback is queued later.

Event loop & task queues:

- **Macro tasks** (task queue): `setTimeout` , `setInterval` , I/O callbacks.
- **Microtasks**: Promise callbacks (`.then`), mutation observers — run before next macro task. Event loop picks tasks from queues and executes them when call stack is empty.

Example to show order:

```
js

console.log('script start');

setTimeout(() => console.log('timeout'), 0);

Promise.resolve().then(() => console.log('promise'));

console.log('script end');

// Output order: script start -> script end -> promise -> timeout
```

Why matters: Helps reason about async behavior, UI freezes, and subtle timing bugs.

Suggested exercises (progression)

1. Implement a small app that uses a module for data handling and a factory for creating UI components.
2. Add an observer-based notification center to decouple modules.
3. Optimize the app: add debounce to heavy input, lazy-load images, and measure reflows.
4. Refactor internals to use pure functions where appropriate; write tests for pure parts.
5. Implement a simple router with code-splitting using dynamic `import()` (if using bundler/packager).

Summary / Quick cheatsheet

- **Module Pattern (IIFE)**: encapsulation and private state. Good for singletons.
- **Revealing Module**: explicitly exposes public API mapped to private functions.
- **Factory**: produce multiple instances; use closures for private fields.
- **Observer**: decouple publisher/subscriber; great for notifications.
- **Debounce**: wait for “pause” — good for search input.
- **Throttle**: limit frequency — good for scroll/resize.
- **IntersectionObserver**: modern efficient lazy-loading and infinite scroll.
- **Code splitting**: load code only when needed (`import()`).
- **Avoid reflows**: batch DOM reads/writes; prefer transforms for animation.
- **Memory leaks**: clear timers and listeners; avoid dangling references.
- **Pure functions**: easier to test and reason about.

- **FP pipeline:** compose map/filter/reduce for transformations.
- **Separation of concerns:** keep logic separate from rendering.
- **Event loop:** microtasks before macrotasks; promises run before timeouts when stack clears.

If you want, I can:

- Convert the above into a printable one-page cheatsheet (PDF),
- Produce step-by-step exercises with starter code for each practice item (e.g., live-search repo),
- Or implement one of the code examples into a runnable snippet you can paste into an HTML file. Which would help you practice fastest?

Sheryians Presents

JavaScript Interview **Essentials**

1. What is hoisting?

JavaScript moves function and variable declarations to the top of their scope before running the code.

👉 That's why you can use a function before it's written.

2. Difference between == and ===

- == checks value only (does type conversion).
- === checks value + type (strict comparison).

👉 Always use === for accuracy.

3. What is a closure? Use case?

A function that remembers the variables from its outer scope even after the outer function has finished.

👉 Used in private variables, currying, and factory functions.

4. What is Event Delegation?

Instead of adding listeners to many child elements, add one to the parent and handle events using `event.target`.

👉 Improves performance and handles dynamic elements.

5. Shallow vs Deep Copy

- Shallow Copy → Only top-level is copied; nested objects still linked.
- Deep Copy → Full independent clone, including nested values.

6. Call, Apply, Bind

All three set this manually.

- `call()` → runs function with args
- `apply()` → same as call but with array
- `bind()` → returns new function with this fixed

7. this in arrow vs normal functions

- Arrow functions don't have their own this → they use the parent's.
- Normal functions get this based on how they're called.

8. Prototype and Inheritance

All JS objects inherit from a prototype.

You can share methods across objects using prototype chaining.

9. Difference: map, reduce, filter

- map() → transforms every item → returns new array
- filter() → removes items → returns new array
- reduce() → turns array into a single value

10. How does async/await work internally?

It's just a prettier way to write Promises.

Behind the scenes, it's still using .then() and a Promise chain.

11. Difference between null and undefined

- undefined → variable declared but not assigned
- null → intentional empty value assigned by you

12. Debounce vs Throttle

- Debounce → runs a function after pause in activity
- Throttle → runs a function every X ms, no matter

how often event fires

13. Memory Leaks in JS

When something (like a timer or event listener) keeps using memory even when it's no longer needed → app gets slow over time.

14. Event Loop Phases

Controls how JS handles tasks.

- Call stack runs sync code
- Web APIs handle async (like setTimeout)
- Event loop moves ready callbacks to stack when it's

empty